```
import { useState, useEffect } from "react";
import {
  createUserWithEmailAndPassword,
  signInWithEmailAndPassword,
  signOut,
  onAuthStateChanged,
  sendPasswordResetEmail,
} from "firebase/auth";
import {
  collection, addDoc, getDocs, doc, setDoc, getDoc,
  query, orderBy, where, onSnapshot
} from "firebase/firestore";
import { auth, db } from "./firebase";

// ─── CONSTANTS ──────────────────────────────────────────────────────────────

const PRATICIENNE_EMAIL = "lamenaturo@gmail.com";

const TRACKING_ITEMS = [
  { key: "complements", label: "Mes compléments", icon: "💊", question: "Tu as
pris tes compléments cette semaine ?" },
  { key: "sommeil", label: "Sommeil", icon: "🌙", question: "Comment tu as dormi
?" },
  { key: "cycle", label: "Cycle & hormones", icon: "🌸", question: "Où en es-tu
dans ton cycle ?" },
  { key: "digestion", label: "Digestion", icon: "🌿", question: "Comment était ta
digestion ?" },
  { key: "energie", label: "Énergie", icon: "⚡", question: "Ton niveau d'énergie
?" },
  { key: "douleurs", label: "Douleurs", icon: "🔥", question: "Des douleurs cette
semaine ?" },
  { key: "humeur", label: "Humeur", icon: "🌊", question: "Tu étais comment
émotionnellement ?" },
  { key: "alimentation", label: "Alimentation", icon: "🥗", question: "Tu as mangé
comment cette semaine ?" },
];

const SCALE = [
  { v: 1, label: "Vraiment pas top", color: "#C4614A" },
  { v: 2, label: "Difficile", color: "#D4906A" },
  { v: 3, label: "Moyen", color: "#C8B86A" },
  { v: 4, label: "Bien", color: "#7BAF8C" },
  { v: 5, label: "Super bien", color: "#5BA08A" },
];
```

```
const C = {
  bg: "#0c0f0e",
  surface: "rgba(255,255,255,0.04)",
  border: "rgba(255,255,255,0.08)",
  accent: "#7EC8A0",
  accentDim: "rgba(126,200,160,0.15)",
  accentBorder: "rgba(126,200,160,0.3)",
  warm: "#C8956C",
  warmDim: "rgba(200,149,108,0.12)",
  text: "rgba(255,255,255,0.88)",
  textMid: "rgba(255,255,255,0.45)",
  textDim: "rgba(255,255,255,0.22)",
};

const inputStyle = {
  width: "100%", background: C.surface, border: `1px solid ${C.border}`,
  borderRadius: 10, padding: "11px 14px", color: C.text,
  fontFamily: "'DM Sans', sans-serif", fontSize: 14, outline: "none",
  boxSizing: "border-box",
};

const btn = (variant = "primary") => ({
  padding: "11px 22px", borderRadius: 10, border: "none", cursor: "pointer",
  fontFamily: "'DM Sans', sans-serif", fontWeight: 600, fontSize: 14,
  transition: "all 0.2s",
  ...(variant === "primary" ? { background: C.accent, color: "#0c0f0e" } :
      variant === "warm" ? { background: C.warm, color: "#0c0f0e" } :
      { background: C.surface, color: C.textMid, border: `1px solid ${C.border}`
}),
});

// ─── CHART ──────────────────────────────────────────────────────────────

function EvolutionChart({ entries }) {
  const [activeKey, setActiveKey] = useState("_avg");
  if (!entries || entries.length < 2) return null;

  const W = 560, H = 180, PAD = { top: 16, right: 16, bottom: 32, left: 28 };
  const innerW = W - PAD.left - PAD.right;
  const innerH = H - PAD.top - PAD.bottom;
  const n = entries.length;

  const getAvg = (e) => {
    const vals = TRACKING_ITEMS.map(i => e.scores?.[i.key]).filter(Boolean);
    return vals.length ? vals.reduce((a, b) => a + b, 0) / vals.length : null;
  };
```

```
  const getSeries = (key) =>
    entries.map(e => key === "_avg" ? getAvg(e) : (e.scores?.[key] ?? null));

  const xPos = (i) => PAD.left + (i / (n - 1)) * innerW;
  const yPos = (v) => PAD.top + innerH - ((v - 1) / 4) * innerH;

  const makePath = (data) => {
    const pts = data.map((v, i) => v !== null ? [xPos(i), yPos(v)] :
null).filter(Boolean);
    if (pts.length < 2) return "";
    return pts.reduce((acc, [x, y], i) => {
      if (i === 0) return `M ${x} ${y}`;
      const [px, py] = pts[i - 1];
      const cx = (px + x) / 2;
      return `${acc} C ${cx} ${py} ${cx} ${y} ${x} ${y}`;
    }, "");
  };

  const seriesColor = activeKey === "_avg" ? C.accent : "#C8956C";
  const data = getSeries(activeKey);
  const path = makePath(data);

  return (
    <div style={{ background: C.surface, borderRadius: 14, border: `1px solid
${C.border}`, padding: 18, marginBottom: 20 }}>
      <div style={{ color: C.textMid, fontSize: 11, textTransform: "uppercase",
letterSpacing: 1, marginBottom: 12 }}>Évolution dans le temps</div>
      <div style={{ display: "flex", flexWrap: "wrap", gap: 6, marginBottom: 16
}}>
        <button onClick={() => setActiveKey("_avg")} style={{
          padding: "4px 12px", borderRadius: 20, border: "none", cursor:
"pointer",
          fontFamily: "'DM Sans', sans-serif", fontSize: 12, fontWeight: 600,
          background: activeKey === "_avg" ? C.accent : "rgba(255,255,255,0.06)",
          color: activeKey === "_avg" ? "#0c0f0e" : C.textMid,
        }}>Score global</button>
        {TRACKING_ITEMS.map(item => (
          <button key={item.key} onClick={() => setActiveKey(item.key)} style={{
            padding: "4px 12px", borderRadius: 20, border: activeKey === item.key
? `1px solid ${C.warm}55` : "1px solid transparent",
            cursor: "pointer", fontFamily: "'DM Sans', sans-serif", fontSize: 12,
            background: activeKey === item.key ? "rgba(200,149,108,0.15)" :
"rgba(255,255,255,0.04)",
            color: activeKey === item.key ? C.warm : C.textDim,
          }}>{item.icon} {item.label}</button>
        ))}
```

```
      </div>
      <div style={{ overflowX: "auto" }}>
        <svg viewBox={`0 0 ${W} ${H}`} style={{ width: "100%", minWidth: 300,
display: "block" }}>
          {[1,2,3,4,5].map(v => (
            <g key={v}>
              <line x1={PAD.left} x2={W-PAD.right} y1={yPos(v)} y2={yPos(v)}
stroke="rgba(255,255,255,0.05)" strokeWidth="1" />
              <text x={PAD.left-6} y={yPos(v)+4} textAnchor="end"
fill="rgba(255,255,255,0.2)" fontSize="9">{v}</text>
            </g>
          ))}
          {path && <path d={`${path} L ${xPos(n-1)} ${PAD.top+innerH} L ${xPos(0)}
${PAD.top+innerH} Z`} fill={seriesColor} fillOpacity="0.07" />}
          {path && <path d={path} fill="none" stroke={seriesColor}
strokeWidth="2.5" strokeLinecap="round" />}
          {entries.map((entry, i) => {
            const v = data[i];
            if (v === null) return null;
            return (
              <g key={i}>
                <circle cx={xPos(i)} cy={yPos(v)} r="5" fill={seriesColor} />
                <circle cx={xPos(i)} cy={yPos(v)} r="9" fill={seriesColor}
fillOpacity="0.15" />
                <text x={xPos(i)} y={yPos(v)-12} textAnchor="middle" fill=
{seriesColor} fontSize="10" fontWeight="700">{v.toFixed(1)}</text>
                <text x={xPos(i)} y={H-4} textAnchor="middle"
fill="rgba(255,255,255,0.25)" fontSize="9">S{i+1}</text>
              </g>
            );
          })}
        </svg>
      </div>
    </div>
  );
}

// ─── SCORE DOT ──────────────────────────────────────────────────────────

function ScoreDot({ value, size = 36 }) {
  const s = SCALE.find(x => x.v === value);
  return (
    <div style={{
      width: size, height: size, borderRadius: "50%",
      background: s ? s.color : C.surface,
      display: "flex", alignItems: "center", justifyContent: "center",
      color: s ? "#0c0f0e" : C.textDim, fontWeight: 800, fontSize: size * 0.38,
```

```
flexShrink: 0,
    }}>{value || "-"}</div>
  );
}

// ─── AUTH SCREEN ──────────────────────────────────────────────────────────

function AuthScreen({ onLogin }) {
  const [mode, setMode] = useState("login");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [prenom, setPrenom] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const [resetSent, setResetSent] = useState(false);

  const handleForgotPassword = async () => {
    if (!email) { setError("Entre ton email d'abord."); return; }
    setError(""); setLoading(true);
    try {
      await sendPasswordResetEmail(auth, email);
      setResetSent(true);
    } catch (e) {
      setError("Email introuvable.");
    }
    setLoading(false);
  };

  const handleLogin = async () => {
    setError(""); setLoading(true);
    try {
      const cred = await signInWithEmailAndPassword(auth, email, password);
      const userDoc = await getDoc(doc(db, "users", cred.user.uid));
      const role = email === PRATICIENNE_EMAIL ? "praticienne" : "cliente";
      onLogin({ uid: cred.user.uid, email, prenom: userDoc.data()?.prenom ||
prenom, role });
    } catch (e) {
      setError("Email ou mot de passe incorrect.");
    }
    setLoading(false);
  };

  const handleRegister = async () => {
    setError(""); setLoading(true);
    if (!email || !password || !prenom) { setError("Remplis tous les champs.");
setLoading(false); return; }
    try {
```

```
      const cred = await createUserWithEmailAndPassword(auth, email, password);
      await setDoc(doc(db, "users", cred.user.uid), {
        prenom, email, role: "cliente", createdAt: new Date().toISOString()
      });
      onLogin({ uid: cred.user.uid, email, prenom, role: "cliente" });
    } catch (e) {
      if (e.code === "auth/email-already-in-use") setError("Un compte existe déjà avec cet email.");
      else if (e.code === "auth/weak-password") setError("Mot de passe trop court (6 caractères minimum).");
      else setError("Erreur lors de la création du compte.");
    }
    setLoading(false);
  };

  return (
    <div style={{ minHeight: "100vh", display: "flex", alignItems: "center", justifyContent: "center", padding: 20, background: `radial-gradient(ellipse at 30% 20%, rgba(126,200,160,0.07) 0%, transparent 55%), ${C.bg}` }}>
      <div style={{ width: "100%", maxWidth: 400 }}>
        <div style={{ textAlign: "center", marginBottom: 36 }}>
          <div style={{ display: "inline-flex", alignItems: "center", gap: 10, marginBottom: 8 }}>
            <div style={{ width: 40, height: 40, borderRadius: 12, background: C.accentDim, border: `1px solid ${C.accentBorder}`, display: "flex", alignItems: "center", justifyContent: "center", fontSize: 20 }}>🌿</div>
            <span style={{ fontFamily: "'Playfair Display', Georgia, serif", fontSize: 22, color: C.text, fontWeight: 700 }}>meije.naturo</span>
          </div>
          <p style={{ color: C.textDim, fontSize: 13 }}>Ton espace de suivi personnalisé</p>
        </div>

        <div style={{ background: C.surface, borderRadius: 16, border: `1px solid ${C.border}`, padding: 28 }}>
          <div style={{ display: "flex", gap: 4, background: "rgba(255,255,255,0.04)", borderRadius: 8, padding: 3, marginBottom: 24 }}>
            {["login", "register"].map(m => (
              <button key={m} onClick={() => { setMode(m); setError(""); }} style={{
                flex: 1, padding: "8px 0", borderRadius: 6, border: "none", cursor: "pointer",
                fontFamily: "'DM Sans', sans-serif", fontSize: 13, fontWeight: 600,
                background: mode === m ? C.accent : "transparent",
                color: mode === m ? "#0c0f0e" : C.textMid,
              }}>{m === "login" ? "Se connecter" : "Créer un compte"}</button>
```

```
            ))}
          </div>

          <div style={{ display: "flex", flexDirection: "column", gap: 12 }}>
            {mode === "register" && (
              <div>
                <label style={{ color: C.textDim, fontSize: 11, textTransform:
"uppercase", letterSpacing: 1, display: "block", marginBottom: 5 }}>Prénom</label>
                <input value={prenom} onChange={e => setPrenom(e.target.value)}
placeholder="Ton prénom" style={inputStyle} />
              </div>
            )}
            <div>
              <label style={{ color: C.textDim, fontSize: 11, textTransform:
"uppercase", letterSpacing: 1, display: "block", marginBottom: 5 }}>Email</label>
              <input value={email} onChange={e => setEmail(e.target.value)}
placeholder="ton@email.fr" type="email" style={inputStyle} />
            </div>
            <div>
              <label style={{ color: C.textDim, fontSize: 11, textTransform:
"uppercase", letterSpacing: 1, display: "block", marginBottom: 5 }}>Mot de
passe</label>
              <input value={password} onChange={e => setPassword(e.target.value)}
placeholder="••••••••" type="password" style={inputStyle} />
            </div>
            {error && <div style={{ color: "#D4826A", fontSize: 13, background:
"rgba(212,130,106,0.1)", borderRadius: 8, padding: "8px 12px" }}>{error}</div>}
            {resetSent && <div style={{ color: C.accent, fontSize: 13, background:
C.accentDim, borderRadius: 8, padding: "8px 12px" }}>✓ Email de réinitialisation
envoyé ! Vérifie ta boîte mail.</div>}
            <button onClick={mode === "login" ? handleLogin : handleRegister}
disabled={loading} style={{ ...btn("primary"), marginTop: 4 }}>
              {loading ? "..." : mode === "login" ? "Se connecter" : "Créer mon
compte"}
            </button>
            {mode === "login" && (
              <button onClick={handleForgotPassword} disabled={loading} style={{
background: "none", border: "none", color: C.textDim, fontSize: 12, cursor:
"pointer", textDecoration: "underline", fontFamily: "'DM Sans', sans-serif",
padding: 0, textAlign: "center" }}>
                Mot de passe oublié ?
              </button>
            )}
          </div>
        </div>
      </div>
    </div>
```

```
  );
}

// ─── CLIENTE SPACE ──────────────────────────────────────────────────────────

function ClienteSpace({ user, onLogout }) {
  const [entries, setEntries] = useState([]);
  const [messages, setMessages] = useState([]);
  const [view, setView] = useState("home");
  const [form, setForm] = useState({ scores: {}, humeur_libre: "", confidences: ""
});
  const [saved, setSaved] = useState(false);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const q = query(collection(db, "entries"), where("userUid", "==", user.uid),
orderBy("date", "asc"));
    const unsub = onSnapshot(q, snap => {
      setEntries(snap.docs.map(d => ({ id: d.id, ...d.data() })));
      setLoading(false);
    });
    const qm = query(collection(db, "messages"), where("toUid", "==", user.uid),
orderBy("date", "asc"));
    const unsub2 = onSnapshot(qm, snap => setMessages(snap.docs.map(d => ({ id:
d.id, ...d.data() }))));
    return () => { unsub(); unsub2(); };
  }, [user.uid]);

  const getWeekLabel = () => {
    const now = new Date();
    const start = new Date(now); start.setDate(now.getDate() - now.getDay() + 1);
    return `Semaine du ${start.getDate()}
${["jan","fév","mar","avr","mai","jun","jul","aoû","sep","oct","nov","déc"]
[start.getMonth()]}`;
  };

  const submitEntry = async () => {
    await addDoc(collection(db, "entries"), {
      userUid: user.uid,
      userEmail: user.email,
      userPrenom: user.prenom,
      weekLabel: getWeekLabel(),
      date: new Date().toISOString(),
      scores: form.scores,
      humeur_libre: form.humeur_libre,
      confidences: form.confidences,
    });
```

```
    setForm({ scores: {}, humeur_libre: "", confidences: "" });
    setSaved(true);
    setTimeout(() => { setSaved(false); setView("home"); }, 1800);
  };

  const lastMessage = messages[messages.length - 1];

  if (loading) return <div style={{ color: C.textDim, padding: 40, textAlign:
"center" }}>Chargement...</div>;

  return (
    <div style={{ minHeight: "100vh", background: `radial-gradient(ellipse at 20%
0%, rgba(126,200,160,0.06) 0%, transparent 55%), ${C.bg}`, padding: "28px 20px",
fontFamily: "'DM Sans', sans-serif" }}>
      <div style={{ maxWidth: 600, margin: "0 auto" }}>
        <div style={{ display: "flex", justifyContent: "space-between",
alignItems: "flex-start", marginBottom: 28 }}>
          <div>
            <div style={{ color: C.accent, fontSize: 11, textTransform:
"uppercase", letterSpacing: 1.5, marginBottom: 4 }}>meije.naturo</div>
            <h1 style={{ fontFamily: "'Playfair Display', Georgia, serif",
fontSize: 26, color: C.text, fontWeight: 700 }}>Bonjour {user.prenom} 🌿</h1>
          </div>
          <button onClick={onLogout} style={{ ...btn("ghost"), fontSize: 12,
padding: "7px 14px" }}>Déconnexion</button>
        </div>

        {lastMessage && (
          <div style={{ background: C.warmDim, border: `1px solid
rgba(200,149,108,0.25)`, borderRadius: 14, padding: 18, marginBottom: 20 }}>
            <div style={{ color: C.warm, fontSize: 11, textTransform: "uppercase",
letterSpacing: 1, marginBottom: 8 }}>Message de Meije</div>
            <p style={{ color: C.text, fontSize: 15, lineHeight: 1.6 }}>
{lastMessage.text}</p>
            <div style={{ color: C.textDim, fontSize: 11, marginTop: 8 }}>{new
Date(lastMessage.date).toLocaleDateString("fr-FR", { day: "numeric", month: "long"
})}</div>
          </div>
        )}

        {view === "home" && (
          <div style={{ display: "flex", flexDirection: "column", gap: 12 }}>
            <button onClick={() => setView("new")} style={{ background:
C.accentDim, border: `1px solid ${C.accentBorder}`, borderRadius: 14, padding:
"20px 22px", cursor: "pointer", textAlign: "left", color: C.text, display: "flex",
alignItems: "center", justifyContent: "space-between" }}>
              <div>
```

```
                <div style={{ color: C.accent, fontWeight: 700, fontSize: 16,
marginBottom: 3 }}>Remplir mon suivi de la semaine</div>
                <div style={{ color: C.textMid, fontSize: 13 }}>{getWeekLabel()}
</div>
              </div>
              <span style={{ fontSize: 22 }}>→</span>
            </button>
            {entries.length > 0 && (
              <button onClick={() => setView("history")} style={{ background:
C.surface, border: `1px solid ${C.border}`, borderRadius: 14, padding: "18px
22px", cursor: "pointer", textAlign: "left", color: C.text, display: "flex",
alignItems: "center", justifyContent: "space-between" }}>
                <div>
                  <div style={{ fontWeight: 600, fontSize: 15, marginBottom: 3
}}>Voir mon historique</div>
                  <div style={{ color: C.textMid, fontSize: 13 }}>{entries.length}
semaine{entries.length > 1 ? "s" : ""} enregistrée{entries.length > 1 ? "s" : ""}
</div>
                </div>
                <span style={{ fontSize: 20 }}>→</span>
              </button>
            )}
          </div>
        )}

        {view === "new" && (
          <div>
            <button onClick={() => setView("home")} style={{ ...btn("ghost"),
fontSize: 12, padding: "6px 14px", marginBottom: 20 }}>← Retour</button>
            <h2 style={{ fontFamily: "'Playfair Display', Georgia, serif",
fontSize: 20, color: C.text, marginBottom: 6 }}>Ton suivi de la semaine</h2>
            <p style={{ color: C.textMid, fontSize: 13, marginBottom: 24 }}>
{getWeekLabel()} · Sois honnête, c'est juste pour toi et Meije 🌿</p>

            {TRACKING_ITEMS.map(item => (
              <div key={item.key} style={{ marginBottom: 20 }}>
                <div style={{ display: "flex", alignItems: "center", gap: 8,
marginBottom: 10 }}>
                  <span style={{ fontSize: 18 }}>{item.icon}</span>
                  <span style={{ color: C.text, fontWeight: 600, fontSize: 15 }}>
{item.question}</span>
                </div>
                <div style={{ display: "flex", gap: 8, flexWrap: "wrap" }}>
                  {SCALE.map(s => (
                    <button key={s.v} onClick={() => setForm(f => ({ ...f, scores:
{ ...f.scores, [item.key]: s.v } }))} style={{
                      padding: "8px 14px", borderRadius: 20,
```

```
                    border: `2px solid ${form.scores[item.key] === s.v ? s.color
: C.border}`,
                    background: form.scores[item.key] === s.v ? s.color + "22" :
"transparent",
                    color: form.scores[item.key] === s.v ? s.color : C.textMid,
                    cursor: "pointer", fontSize: 13, fontWeight:
form.scores[item.key] === s.v ? 700 : 400,
                    fontFamily: "'DM Sans', sans-serif",
                  }}>{s.v} — {s.label}</button>
                ))}
              </div>
            </div>
          ))}

          <div style={{ marginBottom: 16 }}>
            <label style={{ color: C.textDim, fontSize: 11, textTransform:
"uppercase", letterSpacing: 1, display: "block", marginBottom: 6 }}>Comment tu te
sens globalement ?</label>
            <textarea value={form.humeur_libre} onChange={e => setForm(f => ({
...f, humeur_libre: e.target.value }))}
              placeholder="Fatiguée, stressée, plutôt bien..." rows={3} style={{
...inputStyle, resize: "vertical" }} />
          </div>
          <div style={{ marginBottom: 24 }}>
            <label style={{ color: C.textDim, fontSize: 11, textTransform:
"uppercase", letterSpacing: 1, display: "block", marginBottom: 6 }}>Confidences
pour Meije</label>
            <textarea value={form.confidences} onChange={e => setForm(f => ({
...f, confidences: e.target.value }))}
              placeholder="Tout ce que tu veux partager avant votre RDV..."
rows={4} style={{ ...inputStyle, resize: "vertical" }} />
          </div>

          {saved
            ? <div style={{ background: C.accentDim, border: `1px solid
${C.accentBorder}`, borderRadius: 10, padding: 14, color: C.accent, fontWeight:
600, textAlign: "center" }}>✓ Suivi enregistré !</div>
            : <button onClick={submitEntry} style={{ ...btn("primary"), width:
"100%" }}>Envoyer mon suivi à Meije</button>
          }
        </div>
      )}

      {view === "history" && (
        <div>
          <button onClick={() => setView("home")} style={{ ...btn("ghost"),
fontSize: 12, padding: "6px 14px", marginBottom: 20 }}>← Retour</button>
```

```
            <h2 style={{ fontFamily: "'Playfair Display', Georgia, serif",
fontSize: 20, color: C.text, marginBottom: 20 }}>Mon historique</h2>
            <EvolutionChart entries={entries} />
            {[...entries].reverse().map(entry => {
              const vals = TRACKING_ITEMS.map(i => entry.scores?.
[i.key]).filter(Boolean);
              const avg = vals.length ? (vals.reduce((a, b) => a + b, 0) /
vals.length).toFixed(1) : null;
              return (
                <div key={entry.id} style={{ background: C.surface, borderRadius:
12, border: `1px solid ${C.border}`, padding: 18, marginBottom: 12 }}>
                  <div style={{ display: "flex", justifyContent: "space-between",
alignItems: "center", marginBottom: 12 }}>
                    <div>
                      <div style={{ color: C.text, fontWeight: 600 }}>
{entry.weekLabel}</div>
                      <div style={{ color: C.textDim, fontSize: 12 }}>{new
Date(entry.date).toLocaleDateString("fr-FR", { day: "numeric", month: "long" })}
</div>
                    </div>
                    {avg && <ScoreDot value={Math.round(avg)} size={40} />}
                  </div>
                  <div style={{ display: "flex", flexWrap: "wrap", gap: 8,
marginBottom: 8 }}>
                    {TRACKING_ITEMS.filter(i => entry.scores?.[i.key]).map(i => (
                      <div key={i.key} style={{ display: "flex", alignItems:
"center", gap: 5, background: "rgba(255,255,255,0.04)", borderRadius: 20, padding:
"4px 10px" }}>
                        <span style={{ fontSize: 12 }}>{i.icon}</span>
                        <span style={{ fontSize: 12, color: C.textMid }}>{i.label}
</span>
                        <ScoreDot value={entry.scores[i.key]} size={20} />
                      </div>
                    ))}
                  </div>
                  {entry.confidences && (
                    <div style={{ background: "rgba(255,255,255,0.03)",
borderRadius: 8, padding: 10 }}>
                      <div style={{ color: C.textDim, fontSize: 11, marginBottom:
4 }}>Confidences</div>
                      <p style={{ color: C.textMid, fontSize: 13, lineHeight: 1.6
}}>{entry.confidences}</p>
                    </div>
                  )}
                </div>
              );
            })}
```

```
          </div>
        )}
      </div>
    </div>
  );
}

// ─── PRATICIENNE SPACE ──────────────────────────────────────────────

function PraticicenneSpace({ user, onLogout }) {
  const [clients, setClients] = useState([]);
  const [selected, setSelected] = useState(null);
  const [entries, setEntries] = useState([]);
  const [messages, setMessages] = useState([]);
  const [newMsg, setNewMsg] = useState("");
  const [loading, setLoading] = useState(true);
  const [sending, setSending] = useState(false);

  useEffect(() => {
    const q = query(collection(db, "users"), where("role", "==", "cliente"));
    const unsub = onSnapshot(q, snap => {
      setClients(snap.docs.map(d => ({ uid: d.id, ...d.data() })));
      setLoading(false);
    });
    return unsub;
  }, []);

  const selectClient = (client) => {
    setSelected(client);
    setNewMsg("");
    const q = query(collection(db, "entries"), where("userUid", "==", client.uid),
orderBy("date", "asc"));
    onSnapshot(q, snap => setEntries(snap.docs.map(d => ({ id: d.id, ...d.data()
})));
    const qm = query(collection(db, "messages"), where("toUid", "==", client.uid),
orderBy("date", "asc"));
    onSnapshot(qm, snap => setMessages(snap.docs.map(d => ({ id: d.id, ...d.data()
})));
  };

  const sendMessage = async () => {
    if (!newMsg.trim()) return;
    setSending(true);
    await addDoc(collection(db, "messages"), {
      toUid: selected.uid,
      toEmail: selected.email,
      toPrenom: selected.prenom,
```

```
      fromPrenom: "Meije",
      text: newMsg.trim(),
      date: new Date().toISOString(),
    });
    setNewMsg("");
    setSending(false);
  };

  if (loading) return <div style={{ color: C.textDim, padding: 40, textAlign:
"center" }}>Chargement...</div>;

  return (
    <div style={{ minHeight: "100vh", background: `radial-gradient(ellipse at 80%
10%, rgba(200,149,108,0.07) 0%, transparent 50%), ${C.bg}`, fontFamily: "'DM
Sans', sans-serif" }}>
      <div style={{ maxWidth: 900, margin: "0 auto", padding: "28px 20px" }}>
        <div style={{ display: "flex", justifyContent: "space-between",
alignItems: "flex-start", marginBottom: 28 }}>
          <div>
            <div style={{ color: C.warm, fontSize: 11, textTransform: "uppercase",
letterSpacing: 1.5, marginBottom: 4 }}>Espace praticienne</div>
            <h1 style={{ fontFamily: "'Playfair Display', Georgia, serif",
fontSize: 26, color: C.text, fontWeight: 700 }}>Meije · meije.naturo</h1>
          </div>
          <button onClick={onLogout} style={{ ...btn("ghost"), fontSize: 12,
padding: "7px 14px" }}>Déconnexion</button>
        </div>

        {!selected ? (
          <>
            <h2 style={{ color: C.text, fontSize: 16, fontWeight: 600,
marginBottom: 16 }}>
              {clients.length === 0 ? "Aucune consultante inscrite pour
l'instant." : `${clients.length} consultante${clients.length > 1 ? "s" : ""}`}
            </h2>
            <div style={{ display: "flex", flexDirection: "column", gap: 10 }}>
              {clients.map(c => (
                <button key={c.uid} onClick={() => selectClient(c)} style={{
background: C.surface, border: `1px solid ${C.border}`, borderRadius: 12, padding:
"16px 20px", cursor: "pointer", textAlign: "left", color: C.text, display: "flex",
alignItems: "center", justifyContent: "space-between" }}>
                  <div style={{ display: "flex", alignItems: "center", gap: 14 }}>
                    <div style={{ width: 42, height: 42, borderRadius: "50%",
background: C.accentDim, border: `1px solid ${C.accentBorder}`, display: "flex",
alignItems: "center", justifyContent: "center", color: C.accent, fontWeight: 700,
fontSize: 16 }}>
                      {c.prenom?.[0]?.toUpperCase()}
```

```
                    </div>
                    <div>
                      <div style={{ fontWeight: 600, fontSize: 15 }}>{c.prenom}
</div>
                      <div style={{ color: C.textDim, fontSize: 12 }}>{c.email}
</div>
                    </div>
                  </div>
                  <span style={{ color: C.textDim, fontSize: 20 }}>→</span>
                </button>
              ))}
            </div>
          </>
        ) : (
          <div>
            <button onClick={() => setSelected(null)} style={{ ...btn("ghost"),
fontSize: 12, padding: "6px 14px", marginBottom: 20 }}>← Retour</button>
            <div style={{ display: "flex", alignItems: "center", gap: 14,
marginBottom: 24 }}>
              <div style={{ width: 50, height: 50, borderRadius: "50%",
background: C.accentDim, border: `1px solid ${C.accentBorder}`, display: "flex",
alignItems: "center", justifyContent: "center", color: C.accent, fontWeight: 700,
fontSize: 20 }}>
                {selected.prenom?.[0]?.toUpperCase()}
              </div>
              <div>
                <h2 style={{ fontFamily: "'Playfair Display', Georgia, serif",
fontSize: 22, color: C.text, fontWeight: 700 }}>{selected.prenom}</h2>
                <div style={{ color: C.textDim, fontSize: 13 }}>{selected.email} ·
{entries.length} suivi{entries.length > 1 ? "s" : ""}</div>
              </div>
            </div>

            <h3 style={{ color: C.textMid, fontSize: 12, textTransform:
"uppercase", letterSpacing: 1, marginBottom: 14 }}>Suivis de semaine</h3>
            {entries.length >= 2 && <EvolutionChart entries={entries} />}
            {entries.length === 0
              ? <div style={{ color: C.textDim, background: C.surface,
borderRadius: 12, padding: 20, marginBottom: 20, fontSize: 14 }}>{selected.prenom}
n'a pas encore rempli de suivi.</div>
              : [...entries].reverse().map(entry => {
                const vals = TRACKING_ITEMS.map(i => entry.scores?.
[i.key]).filter(Boolean);
                const avg = vals.length ? (vals.reduce((a, b) => a + b, 0) /
vals.length).toFixed(1) : null;
                const s = SCALE.find(x => x.v === Math.round(avg));
                return (
```

```
                <div key={entry.id} style={{ background: C.surface,
borderRadius: 12, border: `1px solid ${C.border}`, padding: 18, marginBottom: 12
}}>
                  <div style={{ display: "flex", justifyContent: "space-
between", alignItems: "center", marginBottom: 14 }}>
                    <div>
                      <div style={{ color: C.text, fontWeight: 600 }}>
{entry.weekLabel}</div>
                      <div style={{ color: C.textDim, fontSize: 12 }}>{new
Date(entry.date).toLocaleDateString("fr-FR", { day: "numeric", month: "long",
year: "numeric" })}</div>
                    </div>
                    {avg && <ScoreDot value={Math.round(avg)} size={40} />}
                  </div>
                  <div style={{ display: "grid", gridTemplateColumns:
"repeat(auto-fill, minmax(180px, 1fr))", gap: 8, marginBottom: 12 }}>
                    {TRACKING_ITEMS.filter(i => entry.scores?.[i.key]).map(i =>
{
                      const sc = SCALE.find(x => x.v === entry.scores[i.key]);
                      return (
                        <div key={i.key} style={{ background:
"rgba(255,255,255,0.03)", borderRadius: 8, padding: "8px 12px", display: "flex",
alignItems: "center", justifyContent: "space-between" }}>
                          <span style={{ fontSize: 13, color: C.textMid }}>
{i.icon} {i.label}</span>
                          <div style={{ display: "flex", alignItems: "center",
gap: 5 }}>
                            <span style={{ fontSize: 11, color: sc?.color }}>
{sc?.label}</span>
                            <ScoreDot value={entry.scores[i.key]} size={22} />
                          </div>
                        </div>
                      );
                    })}
                  </div>
                  {entry.humeur_libre && (
                    <div style={{ background: "rgba(126,200,160,0.06)",
borderRadius: 8, padding: "10px 14px", marginBottom: 8 }}>
                      <div style={{ color: C.accent, fontSize: 11, marginBottom:
4 }}>Humeur & émotions</div>
                      <p style={{ color: C.text, fontSize: 14, lineHeight: 1.6
}}>{entry.humeur_libre}</p>
                    </div>
                  )}
                  {entry.confidences && (
                    <div style={{ background: "rgba(200,149,108,0.07)",
borderRadius: 8, padding: "10px 14px" }}>
```

```
                        <div style={{ color: C.warm, fontSize: 11, marginBottom: 4
}}>Confidences</div>
                        <p style={{ color: C.text, fontSize: 14, lineHeight: 1.6
}}>{entry.confidences}</p>
                      </div>
                    )}
                  </div>
                );
              })
            }

            <h3 style={{ color: C.textMid, fontSize: 12, textTransform:
"uppercase", letterSpacing: 1, marginBottom: 14, marginTop: 28 }}>Ton message à
{selected.prenom}</h3>
            {messages.length > 0 && (
              <div style={{ display: "flex", flexDirection: "column", gap: 8,
marginBottom: 16 }}>
                {messages.map(m => (
                  <div key={m.id} style={{ background: C.warmDim, border: `1px
solid rgba(200,149,108,0.2)`, borderRadius: 10, padding: "12px 16px" }}>
                    <p style={{ color: C.text, fontSize: 14, lineHeight: 1.6 }}>
{m.text}</p>
                    <div style={{ color: C.textDim, fontSize: 11, marginTop: 6 }}>
{new Date(m.date).toLocaleDateString("fr-FR", { day: "numeric", month: "long" })}
</div>
                  </div>
                ))}
              </div>
            )}
            <textarea value={newMsg} onChange={e => setNewMsg(e.target.value)}
              placeholder={`Laisse un message à ${selected.prenom} avant votre
prochain RDV...`} rows={3}
              style={{ ...inputStyle, resize: "vertical", marginBottom: 10 }} />
            <button onClick={sendMessage} disabled={sending || !newMsg.trim()}
style={{ ...btn("warm"), opacity: !newMsg.trim() ? 0.5 : 1 }}>
              {sending ? "Envoi..." : `Envoyer à ${selected.prenom}`}
            </button>
          </div>
        )}
      </div>
    </div>
  );
}

// ─── ROOT ──────────────────────────────────────────────────────────────────

export default function App() {
```

```
  const [user, setUser] = useState(null);
  const [checking, setChecking] = useState(true);

  useEffect(() => {
    const unsub = onAuthStateChanged(auth, async (firebaseUser) => {
      if (firebaseUser) {
        const userDoc = await getDoc(doc(db, "users", firebaseUser.uid));
        const role = firebaseUser.email === PRATICIENNE_EMAIL ? "praticienne" :
"cliente";
        setUser({ uid: firebaseUser.uid, email: firebaseUser.email, prenom:
userDoc.data()?.prenom || "", role });
      } else {
        setUser(null);
      }
      setChecking(false);
    });
    return unsub;
  }, []);

  const handleLogout = async () => { await signOut(auth); setUser(null); };

  if (checking) return (
    <div style={{ minHeight: "100vh", background: "#0c0f0e", display: "flex",
alignItems: "center", justifyContent: "center" }}>
      <div style={{ color: "rgba(255,255,255,0.3)", fontFamily: "'DM Sans', sans-
serif" }}>Chargement...</div>
    </div>
  );

  return (
    <>
      <style>{`
        @import url('https://fonts.googleapis.com/css2?
family=Playfair+Display:wght@400;600;700&family=DM+Sans:wght@400;500;600&display=sw
        * { box-sizing: border-box; margin: 0; padding: 0; }
        body { background: #0c0f0e; }
        input::placeholder, textarea::placeholder { color: rgba(255,255,255,0.2);
}
        input:focus, textarea:focus { border-color: rgba(126,200,160,0.4)
!important; }
        ::-webkit-scrollbar { width: 4px; }
        ::-webkit-scrollbar-thumb { background: rgba(126,200,160,0.2); border-
radius: 2px; }
        button:hover { opacity: 0.88; }
      `}</style>
      {!user
        ? <AuthScreen onLogin={setUser} />
```

```
        : user.role === "praticienne"
          ? <PraticicenneSpace user={user} onLogout={handleLogout} />
          : <ClienteSpace user={user} onLogout={handleLogout} />
      }
    </>
  );
}
```